



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش تمرین دوم درس پردازش زبان‌های طبیعی

N-gram Muisc



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر برادران

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

۳	 مستندات
۳	 ۱- پرسش‌ها
۷	 ۲- برنامه‌نویسی
۱۹	 ۳- خوانش مقاله
۲۲	 ۴- مراجع

مستندات

۱- پرسش‌ها

الف) تفاوت اصلی در محاسبه احتمالات به کمک هموارسازی در مقابل روش discount چیست؟

در smoothing به طور کلی به توزیع احتمال، برای رخدادهای دیده‌نشده هم جرم احتمالی داده می‌شود تا هیچ n -gram یی احتمال صفر نگیرد. (یک هدف کلی)

در discounting ابتدا از شمارش/احتمال رخدادهای دیده‌شده مقدار کمی، کم می‌شود و همان جرم آزادشده به رخدادهای نادیده یا کم‌رخداد اختصاص می‌یابد. (یک ابزار)

پس smoothing یک فرم کلی با هدفی است که کل توزیع احتمالی اصلاح شود تا هیچ رویدادی احتمال صفر نگیرد و discounting یکی از خانواده‌های رایج روش‌های smoothing است که به جای اضافه کردن مستقیم با آزاد کردن بخشی از جرم احتمال، از دیده‌شده‌ها کم می‌کند.

ب) برای هر جمله کم‌ترین مقدار n که بتواند روابط جمله مورد نظر را در یابد، مشخص کنید.

این سؤال کیفی است و یک جواب یکتا ندارد، اما به طور منطقی:

در مدل n -gram، احتمال هر کلمه بر اساس $n-1$ کلمه‌ی قبلی برآورد می‌شود. بنابراین هرچه وابستگی‌های جمله بلندتر و ساختار آن آزادتر باشد، به n بزرگ‌تری نیاز داریم.

**** - من رفتند آسمان کتاب برای چرا فواره ها ****

در اینجا به خاطر آزاد بودن ترتیب و فاصله‌ی زیاد میان وابستگی‌ها و نامعمول بودن ساختار، n کوچک رابطه‌ها را خوب نمی‌گیرد؛ پس معمولاً باید n بزرگ‌تر باشد (حداقل حدود ۴ یا بیشتر). با n کم، وابستگی‌های دور از هم از دست می‌روند.

**** - ای شب از رویای تو رنگین شده ****

ساختار نسبتاً شاعرانه است ولی وابستگی‌های محلی دارد؛ معمولاً trigram یا نهایتاً 4-gram کافی‌تر است. روابطی مثل «رویای تو» و «رنگین شده» نیاز به دیدن دو کلمه‌ی قبلی دارند، و bigram به‌تنهایی context کافی نمی‌دهد.

**** - she suddenly spots a White Rabbit ****

جمله‌ی استاندارد و محلی‌محور است؛ bigram یا trigram معمولاً کافی است. برای رابطه‌هایی مثل adjective-noun و verb-object، trigram بهتر از bigram است. عبارت «a White Rabbit» و ارتباط «spots a White Rabbit» با bigram کامل دیده نمی‌شود، ولی trigram می‌تواند این وابستگی محلی را بگیرد.

**** - eins zwei drei vier fünf sechs sieben ****

دنباله‌ی ترتیبی و بسیار منظم است (شمارشی)؛ bigram هم معمولاً کافی است چون الگو تقریباً زنجیره‌ای و محلی است و هر واژه عمدتاً به واژه‌ی قبلی وابسته است.

نکته: آیا افزایش مقدار n در عمل باعث بهبود نتایج می شود؟ چرا؟

خیر. افزایش n فقط تا جایی مفید است که داده کافی برای برآورد n -gram های بزرگ داشته باشیم و sparsity خیلی شدید نشود و مدل بیش از حد روی الگوهای خاص آموزشی overfit نکند. با بزرگ تر شدن n ، مدل context بیشتری می بیند اما تعداد n -gram های دیده نشده را هم زیاد می کند که سبب sparsity و overfitting بیشتر می شود، پس n بزرگ تر الزاماً بهتر نیست اما هرچه ساختار جمله آزادتر و وابستگی ها دورتر باشند، بسته به شرایط مسئله n بزرگ تری لازم می شود که بهترین n را باید با validation/perplexity انتخاب کرد و در عمل، باید بین قدرت مدل سازی و پوشش آماری تعادل برقرار کرد.

ج) با توجه به پیکره ی متنی زیر، احتمالات خواسته شده را محاسبه کنید.

هر سه جمله در پیکره به یک معنا هستند ولی ترتیب قرارگیری کلمات با دیگری متفاوت و هر یک جمله ای معتبری را می سازند (انعطاف ساختار جملات در زبان روسی) و نشان می دهد که یک مدل n -gram ساده همیشه نمی تواند ساختار عمیق معنایی را به طور کامل درک کند.

پیکره:

- ``<s> my gulyali v lesu </s>``

- ``<s> gulyali my v lesu </s>``

- ``<s> v lesu my gulyali </s>``

احتمال‌ها با ****MLE**** :

- تعداد وقوع‌های my در نقش context برابر ۳ است.
- فقط یک‌بار دنباله‌ی lesu my در پیکره دیده می‌شود، و بعد از آن gulyali آمده است.
- دنباله‌ی my lesu در پیکره اصلاً دیده نشده است.

$$- \text{`p(gulyali | my) =}$$

$$\text{count(my gulyali) / count(my) = } 2/3 \text{`}$$

$$- \text{`p(gulyali | lesu, my) =}$$

$$\text{count(lesu my gulyali) / count(lesu my) = } 1/1 = 1 \text{`}$$

$$- \text{`p(lesu | my) =}$$

$$\text{count(my lesu) / count(my) = } 0/3 = 0 \text{`}$$

۲- برنامه‌نویسی

کد پایتونی این تمرین، یک نوت‌بوک کامل برای اجرا در Colab است که:

- 'music.csv' را می‌خواند.

- n-gram ها را می‌سازد.

- alpha را با validation تنظیم می‌کند.

- perplexity را محاسبه می‌کند.

- پنج قطعه‌ی ناقص را کامل می‌کند.

- یک بار هم آزمایش را با نادیده‌گرفتن 'b' و 'k' تکرار می‌کند.

(۱) طراحی مسئله

در این مسئله، بهترین راه‌حل این است که روی دنباله‌ی نت‌ها یک مدل n-gram با هموارسازی Lidstone بسازیم، بعد با **perplexity** آن را ارزیابی کنیم و برای تکمیل قطعات از بیشترین احتمال شرطی استفاده کنیم.

نتیجه‌های اصلی برای داده‌ی خام این‌هاست:

- **سرگشتگی (perplexity)** هرچه کمتر باشد بهتر است.

- روی داده‌ی خام، در ارزیابی leave-one-out :

- **Unigram** ضعیف‌ترین است، چون اصلاً بافت را نمی‌بیند.

- **Bigram** بهترین تعادل را دارد.

- **Trigram** روی این دیتای کوچک کمی دچار sparsity می‌شود.

• بین دو مقدار خواسته شده ی α ، برای داده ی خام:

◦ **Unigram** : $\alpha=1$ کمی بهتر از $\alpha=0.5$

◦ **Bigram** : $\alpha=0.5$ کمی بهتر از $\alpha=1$

◦ **Trigram** : $\alpha=0.5$ کمی بهتر از $\alpha=1$

• اگر همه ی modifier ها مثل b و k را حذف کنیم، داده فشرده تر و پراکندگی کمتر می شود، پس **perplexity** پایین تر می آید و مدل ها پایدارتر می شوند؛ ولی دقت ظرافت های موسیقایی کمتر می شود.

• قطعه ی **A S B Db** یا در نسخه ی نرمال شده A S B D، برای هر سه مدل معمولاً بدترین خروجی را می دهد، چون هم توالی های قبلی آن کم تکرارترند و هم خودش شامل علامت/حالت کم مشاهده تر است.

• اگر **dastgah** قطعه معلوم باشد، استفاده از آن خیلی کمک می کند، چون توزیع نت ها را به یک زیرفضای کوچک تر و همگن تر محدود می کند اما مشکلش این است که بعضی برچسب ها نامنظم یا ناقص اند (-)، بعضی دستگاه ها داده ی خیلی کمی دارند و برخی قطعات هم مدولاسیون (تغییر گام) دارند. در شکل زیر، برای هر prefix، score هر dastgah را می دهد تا نشان دهد هر fragment بیشتر شبیه کدام دستگاه است. ('homayoon', -6.77) یعنی اگر این prefix را زیر مدل homayoon حساب کنیم، log-probability آن برابر ۶.۷۷- شده است (چون probability ها کمتر از ۱ هستند و log آن ها منفی می شود و کمتر منفی = بهتر / بزرگ تر = محتمل تر).

```
[('S', 'S', 'S', 'S')] [('homayoon', -6.774223886357615), ('dashti', -9.095340958446386), ('shoor', -10.560981648381587), ('mahoor', -10.90916184554497)]
[('G', 'C', 'G', 'C')] [('homayoon', -6.774223886357615), ('mahoor', -7.195787290198207), ('shoor', -7.978742573408493), ('dashti', -10.337054090755167)]
[('C', 'B', 'C', 'D')] [('mahoor', -5.533267863818589), ('homayoon', -6.774223886357615), ('dashti', -9.095340958446386), ('shoor', -9.629423444376641)]
[('A', 'Bb', 'C', 'D')] [('homayoon', -6.774223886357615), ('shoor', -7.709615919223966), ('dashti', -8.51630708465386), ('mahoor', -10.08818129347514)]
[('A', 'S', 'B', 'Db')] [('homayoon', -6.774223886357615), ('shoor', -9.796477529039809), ('dashti', -10.026899162451329), ('mahoor', -10.59900691724113)]
```


۲) پاسخ نظری به سؤال‌ها

الف) کمترین α را چگونه پیدا کنیم؟

برای هر مدل، داده را به train/validation یا بهتر از آن leave-one-song-out تقسیم می‌کنیم، برای چند مقدار α مدل را آموزش می‌دهیم و α ی را انتخاب می‌کنیم که perplexity کم‌تری روی داده‌ی دیده‌نشده بدهد. در این داده، اگر فقط بین $\alpha=1$ و $\alpha=0.5$ مطابق صورت مسئله مقایسه کنیم، $\alpha=0.5$ برای bigram و trigram بهتر است، اما برای unigram مقدار $\alpha=1$ کمی بهتر است.

ب) چرا perplexity معیار خوبی است؟

چون نشان می‌دهد مدل تا چه حد توالی نت‌های واقعی را محتمل می‌داند. هرچه perplexity کمتر، مدل روی داده‌ی آزمون غافلگیری کم‌تری دارد.

پ) چه معیار دیگری مناسب است؟

برای موسیقی، فقط perplexity کافی نیست. چون چند ادامه‌ی مختلف می‌تواند موسیقایی و درست باشد. معیارهای بهتر مکمل:

- Similarity روی فاصله‌های ملودیک،
- Overlap آماری روی n-gram‌های نت،
- ارزیابی انسانی/شنیداری.
- اما برای این تمرین، perplexity طبیعی‌ترین معیار است.

ت) ترتیب عملکرد مدل‌ها چرا این‌طور است؟

روی داده‌ی خام، معمولاً:

bigram > trigram > unigram

علت:

- Unigram بافت را نادیده می‌گیرد.
- Bigram از بافت محلی استفاده می‌کند و هنوز داده‌کافی برای تخمین دارد.
- Trigram با اینکه بافت بیشتری می‌بیند، روی دیتاست کوچک خیلی sparse می‌شود.

ث) کدام مدل توزیع‌هایش بیشتر نماینده‌ی داده است؟

اگر منظور بین α های جدول باشد، $\alpha=0.5$ معمولاً توزیع را کمتر صاف می‌کند و به فراوانی‌های واقعی نزدیک‌تر است؛ یعنی empirical distribution را بهتر حفظ می‌کند.

اگر منظور بین خود مدل‌ها باشد، bigram روی این دیتاست تعادل بهتری بین «نمایندگی داده» و «پراکندگی کم» دارد.

ج) اگر b و k را حذف کنیم چه می‌شود؟

- Vocab کوچک‌تر می‌شود.
- Sparsity کمتر می‌شود.
- Perplexity پایین‌تر می‌آید.

- ولی تمایزهایی مثل Bb و B یا Ak و A از بین می‌رود. یعنی مدل از نظر آماری راحت‌تر می‌شود، اما از نظر موسیقایی خام‌تر.

(چ) کدام قطعه‌ی ناقص بدتر تکمیل می‌شود؟

به‌طور معمول قطعه‌هایی که context کم‌سابقه‌تری دارند، مثل شروع‌های خیلی خاص یا توالی‌هایی با نت‌های نادر، بدتر تکمیل می‌شوند. در این داده تقریباً برای هر سه مدل، A S B Db بدترین است. چون:

- هم Db در داده‌ی آموزش خیلی کم یا اصلاً دیده نشده،
- هم context‌های قبل از آن کمیاب‌اند،
- هم در bigram/trigram به مشکل sparsity می‌خوریم (یعنی مدل داده‌ی کافی برای بعضی context‌ها ندارد).

۳) نقش dastgah در موسیقی

ستون `dastgah` می‌تواند به مدل کمک کند چون به‌جای یادگیری یک مدل عمومی برای همه‌ی قطعات، مدل می‌تواند الگوهای مخصوص هر دستگاه را هم یاد بگیرد. دو راه معمول:

1. ساخت مدل جدا برای هر دستگاه
2. اضافه کردن یک token ویژه‌ی دستگاه در ابتدای دنباله و آموزش یک مدل شرطی

مشکل‌ها:

- تعداد داده‌ها کم است.
- بعضی قطعات '-' دارند.
- بعضی دستگاه‌ها خیلی کم‌نمونه‌اند.
- ممکن است مدل به‌جای الگوی موسیقایی، فقط به دستگاه overfit کند.

جمع‌بندی

خروجی‌های بدست آمده که در ادامه قرار دارد، greedy decoding هستند یعنی هر بار محتمل‌ترین نتِ بعدی انتخاب شده که از نظر اجرای کد اشتباه نیست، اما از نظر کیفیت تکمیل ضعیف است. دلیلش این است که مدل n-gram روی یک پیکره‌ی خیلی کوچک آموزش دیده و در چنین حالتی معمولاً روی یک نتِ پرتکرار مثل C قفل می‌کند و یک رفتار طبیعیِ مدل ساده است. برای همین در ادامه‌ی آن، نسخه‌ی اصلاح‌شده‌ای می‌سازیم که EOS را وسط کار وارد نمی‌کند و برای جلوگیری از چرخه‌های تکراری یک blocking ساده دارد. (در این تمرین LOOCV برای تقسیم‌بندی خوب است چون داده کم است.)

در این تمرین، n-gram یک مدل ساده و آموزشی است و انتظار نمی‌رود ساختار دقیق موسیقی را کامل یاد بگیرد. هدف اصلی، نشان دادن اثر n-gram order، smoothing، alpha و استفاده‌ی اختیاری از dastgah است.

```

=====
ignore_modifiers=False | use_dastgah=False
Number of pieces: 41
Vocabulary size: 14

[n=1] best alpha = 2.0
alpha= 0.1: val perplexity=12.0478
alpha= 0.2: val perplexity=12.0356
alpha= 0.3: val perplexity=12.0264
alpha= 0.4: val perplexity=12.0186
alpha= 0.5: val perplexity=12.0115
alpha=0.75: val perplexity=11.9960
alpha= 1.0: val perplexity=11.9825
alpha= 1.5: val perplexity=11.9591
alpha= 2.0: val perplexity=11.9392
train perplexity (best alpha) = 10.8379

[n=2] best alpha = 1.0
alpha= 0.1: val perplexity=9.7932
alpha= 0.2: val perplexity=9.4689
alpha= 0.3: val perplexity=9.3300
alpha= 0.4: val perplexity=9.2549
alpha= 0.5: val perplexity=9.2110
alpha=0.75: val perplexity=9.1664
alpha= 1.0: val perplexity=9.1658
alpha= 1.5: val perplexity=9.2167
alpha= 2.0: val perplexity=9.2954
train perplexity (best alpha) = 7.5064

[n=3] best alpha = 0.5
alpha= 0.1: val perplexity=11.9744
alpha= 0.2: val perplexity=10.7137
alpha= 0.3: val perplexity=10.2807
alpha= 0.4: val perplexity=10.0973
alpha= 0.5: val perplexity=10.0218
alpha=0.75: val perplexity=10.0239
alpha= 1.0: val perplexity=10.1261
alpha= 1.5: val perplexity=10.3959
alpha= 2.0: val perplexity=10.6639
train perplexity (best alpha) = 5.5472

Perplexity for the required alpha values (1.0 and 0.5):
n=1: alpha=1.0 -> 10.8308, alpha=0.5 -> 10.8285
n=2: alpha=1.0 -> 7.5064, alpha=0.5 -> 7.3073
n=3: alpha=1.0 -> 6.4520, alpha=0.5 -> 5.5472

Fixed-length completions (10 notes) for each prompt and each model:

--- n=1, alpha=2.0 ---
1. S S S S -> C C D C C D C C D C
2. G C G C -> C D C C D C C D C C
3. C B C D -> C C D C C D C C D C
4. A Bb C D -> C C D C C D C C D C
5. A S B Db -> C C D C C D C C D C

--- n=2, alpha=1.0 ---
1. S S S S -> D C D C D C C D C C
2. G C G C -> C D C C D C C D C C

--- n=3, alpha=0.5 ---
1. S S S S -> G Ab Bb Ab S Bb Ab S Bb C
2. G C G C -> C G G F G F G F G F
3. C B C D -> C Bb C D C Bb C D C D
4. A Bb C D -> C Bb C D C Bb C D C D
5. A S B Db -> A A G G F G F Eb D C

=====
ignore_modifiers=True | use_dastgah=False
Number of pieces: 41
Vocabulary size: 8

[n=1] best alpha = 2.0
alpha= 0.1: val perplexity=8.6721
alpha= 0.2: val perplexity=8.6717
alpha= 0.3: val perplexity=8.6713
alpha= 0.4: val perplexity=8.6709
alpha= 0.5: val perplexity=8.6705
alpha=0.75: val perplexity=8.6695
alpha= 1.0: val perplexity=8.6686
alpha= 1.5: val perplexity=8.6667
alpha= 2.0: val perplexity=8.6648
train perplexity (best alpha) = 8.3059

[n=2] best alpha = 2.0
alpha= 0.1: val perplexity=7.0538
alpha= 0.2: val perplexity=6.9959
alpha= 0.3: val perplexity=6.9602
alpha= 0.4: val perplexity=6.9338
alpha= 0.5: val perplexity=6.9128
alpha=0.75: val perplexity=6.8741
alpha= 1.0: val perplexity=6.8472
alpha= 1.5: val perplexity=6.8131
alpha= 2.0: val perplexity=6.7947
train perplexity (best alpha) = 6.1028

[n=3] best alpha = 1.5
alpha= 0.1: val perplexity=8.6573
alpha= 0.2: val perplexity=7.8067
alpha= 0.3: val perplexity=7.4481
alpha= 0.4: val perplexity=7.2507
alpha= 0.5: val perplexity=7.1287
alpha=0.75: val perplexity=6.9741
alpha= 1.0: val perplexity=6.9152
alpha= 1.5: val perplexity=6.9007
alpha= 2.0: val perplexity=6.9368
train perplexity (best alpha) = 5.2415

Perplexity for the required alpha values (1.0 and 0.5):
n=1: alpha=1.0 -> 8.3057, alpha=0.5 -> 8.3057
n=2: alpha=1.0 -> 6.0576, alpha=0.5 -> 6.0396
n=3: alpha=1.0 -> 5.0068, alpha=0.5 -> 4.7137

```

```

Fixed-length completions (10 notes) for each prompt and each model: [n=1] best alpha = 2.0
alpha= 0.1: val perplexity=9.9867
alpha= 0.2: val perplexity=9.9780
alpha= 0.3: val perplexity=9.9727
alpha= 0.4: val perplexity=9.9689
alpha= 0.5: val perplexity=9.9659
alpha=0.75: val perplexity=9.9603
alpha= 1.0: val perplexity=9.9564
alpha= 1.5: val perplexity=9.9514
alpha= 2.0: val perplexity=9.9487
train perplexity (best alpha) = 9.3583

--- n=1, alpha=2.0 ---
1. S S S S -> C C D C C D C C D C
2. G C G C -> C D C C D C C D C C
3. C B C D -> C C D C C D C C D C
4. A B b C D -> C C D C C D C C D C
5. A S B D b -> C C D C C D C C D C

--- n=2, alpha=2.0 ---
1. S S S S -> E D C C D C C D C C
2. G C G C -> C D C C D C C D C C
3. C B C D -> E D C C D C C D C C
4. A B b C D -> E D C C D C C D C C
5. A S B D b -> E D C C D C C D C C

--- n=3, alpha=1.5 ---
1. S S S S -> G A B C D C B C D C
2. G C G C -> C B C D C B C D C B
3. C B C D -> C B C D C B C D C B
4. A B b C D -> C B C D C B C D C B
5. A S B D b -> A F G F G F G F E D

=====
ignore_modifiers=True | use_dastgah=True
Number of pieces: 41
Vocabulary size: 13

[n=2] best alpha = 0.3
alpha= 0.1: val perplexity=7.0024
alpha= 0.2: val perplexity=6.9589
alpha= 0.3: val perplexity=6.9487
alpha= 0.4: val perplexity=6.9508
alpha= 0.5: val perplexity=6.9591
alpha=0.75: val perplexity=6.9931
alpha= 1.0: val perplexity=7.0356
alpha= 1.5: val perplexity=7.1301
alpha= 2.0: val perplexity=7.2282
train perplexity (best alpha) = 5.9662

[n=3] best alpha = 0.5
alpha= 0.1: val perplexity=8.9277
alpha= 0.2: val perplexity=8.2354
alpha= 0.3: val perplexity=8.0063
alpha= 0.4: val perplexity=7.9207

```

```

alpha= 0.2: val perplexity=8.2354
alpha= 0.3: val perplexity=8.0063
alpha= 0.4: val perplexity=7.9207
alpha= 0.5: val perplexity=7.8981
alpha=0.75: val perplexity=7.9557
alpha= 1.0: val perplexity=8.0740
alpha= 1.5: val perplexity=8.3491
alpha= 2.0: val perplexity=8.6188
train perplexity (best alpha) = 5.0945

Perplexity for the required alpha values (1.0 and 0.5):
n=1: alpha=1.0 -> 9.3495, alpha=0.5 -> 9.3466
n=2: alpha=1.0 -> 6.1850, alpha=0.5 -> 6.0334
n=3: alpha=1.0 -> 5.7232, alpha=0.5 -> 5.0945

Fixed-length completions (10 notes) for each prompt and each model:

--- n=1, alpha=2.0 ---
1. S S S S -> C C D C D C C D C C
2. G C G C -> C D C C D C C D C C
3. C B C D -> C C D C D C C D C C
4. A B b C D -> C C D C D C C D C C
5. A S B D b -> C C D C D C C D C C

--- n=2, alpha=0.3 ---
1. S S S S -> E D C C D C C D C C
2. G C G C -> C D C C D C C D C C
3. C B C D -> E D C C D C C D C C
4. A B b C D -> E D C C D C C D C C
5. A S B D b -> F E D E D C C D C C

--- n=3, alpha=0.5 ---
1. S S S S -> G A B A S B C D C B
2. G C G C -> C B C D C B C D C B
3. C B C D -> C B C D C B C D C B
4. A B b C D -> C B C D C B C D C B
5. A S B D b -> A F G F E D C B C D

=====
ignore_modifiers=False | use_dastgah=True
Number of pieces: 41
Vocabulary size: 19

[n=1] best alpha = 2.0
alpha= 0.1: val perplexity=13.6589
alpha= 0.2: val perplexity=13.6367
alpha= 0.3: val perplexity=13.6215
alpha= 0.4: val perplexity=13.6094
alpha= 0.5: val perplexity=13.5991
alpha=0.75: val perplexity=13.5777
alpha= 1.0: val perplexity=13.5603
alpha= 1.5: val perplexity=13.5327
alpha= 2.0: val perplexity=13.5115
train perplexity (best alpha) = 12.1038

[n=2] best alpha = 0.4
alpha= 0.1: val perplexity=9.6271
alpha= 0.2: val perplexity=9.3774
alpha= 0.3: val perplexity=9.3056
alpha= 0.4: val perplexity=9.2921
alpha= 0.5: val perplexity=9.3053
alpha=0.75: val perplexity=9.3890

```

<pre>[n=2] best alpha = 0.4 alpha= 0.1: val perplexity=9.6271 alpha= 0.2: val perplexity=9.3774 alpha= 0.3: val perplexity=9.3056 alpha= 0.4: val perplexity=9.2921 alpha= 0.5: val perplexity=9.3053 alpha=0.75: val perplexity=9.3890 alpha= 1.0: val perplexity=9.5015 alpha= 1.5: val perplexity=9.7474 alpha= 2.0: val perplexity=9.9924 train perplexity (best alpha) = 7.2415 [n=3] best alpha = 0.4 alpha= 0.1: val perplexity=12.3439 alpha= 0.2: val perplexity=11.3489 alpha= 0.3: val perplexity=11.1068 alpha= 0.4: val perplexity=11.0779 alpha= 0.5: val perplexity=11.1342 alpha=0.75: val perplexity=11.4044 alpha= 1.0: val perplexity=11.7198 alpha= 1.5: val perplexity=12.3205 alpha= 2.0: val perplexity=12.8420 train perplexity (best alpha) = 5.7452</pre>	<pre>Perplexity for the required alpha values (1.0 and 0.5): n=1: alpha=1.0 -> 12.0857, alpha=0.5 -> 12.0796 n=2: alpha=1.0 -> 7.7281, alpha=0.5 -> 7.3304 n=3: alpha=1.0 -> 7.3783, alpha=0.5 -> 6.0680 Fixed-length completions (10 notes) for each prompt and each model: --- n=1, alpha=2.0 --- 1. S S S S -> C C D C C D C C D C 2. G C G C -> C D C C D C C D C C 3. C B C D -> C C D C C D C C D C 4. A Bb C D -> C C D C C D C C D C 5. A S B Db -> C C D C C D C C D C --- n=2, alpha=0.4 --- 1. S S S S -> D C C D C C D C D C 2. G C G C -> C D C C D C C D C C 3. C B C D -> C D C C D C C D C C 4. A Bb C D -> C D C C D C C D C C 5. A S B Db -> B C D C D C C D C C --- n=3, alpha=0.4 --- 1. S S S S -> G Ab Bb Ab S Bb Ab S Bb C 2. G C G C -> C G G F G F G F G F 3. C B C D -> C Bb C D C Bb C D C D 4. A Bb C D -> C Bb C D C Bb C D C D 5. A S B Db -> A A G G F G F Eb D C</pre>
--	---

- مدل **unigram** اصلاً به **prefix** توجه نمی‌کند و همیشه نتِ پر تکرارتر را پیشنهاد می‌دهد. در داده‌ی خام، این نت C بوده، برای همین همه‌جا C تکرار شده‌است و گاهی هم D تکرار شد زیرا برای جلوگیری از چرخه‌های تکراری یک **blocking** ساده ساختیم.

- **Number of pieces: 41** یعنی در فایل **music.csv**، ۴۱ قطعه/ردیف برای آموزش مدل وجود داشته است. پس مدل از ۴۱ نمونه یاد می‌گیرد.

- **Vocabulary size: 14** یعنی تعداد توکن‌های یکتای موجود در داده بعد از

پیش‌پردازش برابر ۱۴ است و مدل در این حالت ۱۴ نشانه‌ی متفاوت دیده است.

مثلاً نت‌هایی از این جنس: A, Bb, B, C, Db, D, E, F, G, Ab, S, ...

○ هرچه vocabulary بزرگ‌تر باشد:

▪ مدل آزادی بیشتری دارد،

▪ اما داده‌ی بیشتری هم برای خوب تخمین‌زدن لازم دارد.

○ در حالت `ignore_modifiers=True` این عدد به ۸ رسیده، چون Bb

و B یا Db و D به نت پایه تبدیل شده‌اند و تعداد نشانه‌ها کمتر شده است.

○ در حالت `use_dastgah=True` هم عدد بیشتر می‌شود، چون برای هر

دستگاه یک توکن ویژه اضافه می‌شود.

- α همان **Lidstone smoothing parameter** است.

$$P(w|h) = \frac{\text{count}(h, w) + \alpha}{\text{count}(h) + \alpha|V|}$$

اگر یک n-gram دیده نشده باشد، باز هم احتمال آن صفر نمی‌شود.

α تعیین می‌کند چقدر احتمال به رخ داده‌های ندیده اضافه شود.

از نظر نظری، هر عدد مثبت می‌تواند باشد ولی در عمل معمولاً روی یک grid

امتحان می‌کنند.

α کوچک = مدل تیزتر و نزدیک‌تر به داده‌ی خام و مشکل در داده‌های دیده‌نشده

α بزرگ‌تر = مدل نرم‌تر و یکنواخت‌تر و نزدیک به uniform و یکنواختی ارزش

Laplace smoothing یعنی $\alpha = 1$

Jeffreys-Perks یعنی $\alpha = 0.5$

- چرا **train perplexity** از **validation** کمتر یا نزدیک تر است؟
این عدد روی همان داده‌ی آموزشی حساب شده است که چون مدل روی همان داده آموزش دیده، معمولاً این عدد از **validation** بهتر است.
اگر اختلاف خیلی زیاد باشد، یعنی مدل احتمالاً **overfit** کرده است.
- این مدل دانش موسیقایی واقعی ندارد؛ فقط می‌داند بعد از چه توالی‌ای چه چیزی بیشتر دیده شده است. خروجی قطعات ناقص یگانه و قطعی نیست؛ چون مدل‌های **n-gram** ممکن است چند ادامه‌ی معقول داشته باشند.
- چرا نت‌های کرن‌دار و بمل‌دار کم تولید شده‌اند؟
 - ۱- در داده کم‌تر دیده شده‌اند و **n-gram** فقط از فراوانی‌ها یاد می‌گیرد. اگر **Bb** کم آمده باشد، احتمال خروجی‌اش هم کم می‌شود.
 - ۲- **Smoothing** فقط جلوی صفر شدن را می‌گیرد، نه این که **rare token** را محبوب کند یعنی **Bb** احتمال صفر نمی‌گیرد، اما هنوز از نت‌های پرتکرار ضعیف‌تر است.
 - ۳- در حالت **ignore_modifiers=True** حذف می‌شوند. پس در آن **run** نباید تولید شوند.
 - ۴- دیتاست کوچک است و مدل معمولاً به سمت نت‌های عمومی‌تر مثل **C** و **D** می‌رود.
- در **n-gram**‌های بالاتر، الفای کمتری لازم بوده زیرا **smoothing** زیادتر باعث شده احتمالات خیلی صاف شوند و الگوی دقیق توالی‌ها ضعیف‌تر شود.

$$\text{count}(h, w) \ll \alpha$$

$$P(w|h) \approx \frac{\alpha}{\alpha V} = \frac{1}{V}$$

اگر α خیلی بزرگ شود یعنی همه‌ی نت‌ها تقریباً احتمال برابر می‌گیرند و اطلاعات واقعی داده را فراموش می‌کند.

- با توجه به ذات این مسئله، بهترین روش ارزیابی مکمل، ارزیابی انسانی زوجی (**pairwise human evaluation**) است؛ یعنی دو خروجی مدل را کنار هم بگذاریم و از فردی آشنا با موسیقی بخواهیم بگویند کدام یک از نظر طبیعی بودن ملودی، پیوستگی، و سازگاری با سبک قطعه بهتر است. زیرا: چون در این مسئله فقط یک جواب «قطعی و یکتا» وجود ندارد. برای یک دنباله‌ی ناقص، چند ادامه‌ی مختلف می‌توانند همگی از نظر موسیقایی قابل قبول باشند. بنابراین معیارهایی مثل perplexity فقط می‌گویند مدل چقدر ادامه‌ها را از نظر آماری خوب حدس زده، اما لزوماً نمی‌گویند خروجی خوش‌صدا، منطقی و موسیقایی هست یا نه.

- اگر بخواهیم یک معیار عددی هم کنار آن پیشنهاد دهیم، / edit distance / note-level accuracy یا **top-k next-note accuracy** می‌تواند مفید باشد؛ چون نشان می‌دهد مدل چقدر به ادامه‌ی واقعی نزدیک شده است. اما برای این تمرین، از نظر مفهومی، ارزیابی انسانی مقایسه‌ای مناسب‌تر از همه است، چون کیفیت موسیقایی را بهتر از یک عدد آماری ساده نشان می‌دهد.

۳- خوانش مقاله

(۱) هدف مقاله چیست؟

هدف مقاله ارائه‌ی یک سیستم ترجمه‌ی ماشینی عصبی در مقیاس تولید و سرتاسری است که هم دقت بالاتری نسبت به سیستم‌های phrase-based داشته باشد و هم مشکلاتی مثل سرعت پایین و کندی آموزش و inference، ضعف در کلمات نادر، پوشش ناقص ورودی و ترجمه را بهتر حل کند.

(۲) مشکلات NMT‌های پیشین چه بود؟

مقاله سه مشکل اصلی را برجسته می‌کند:

- سرعت پایین آموزش و inference،
- ضعف در برخورد با کلمات نادر (روش‌های copy یا attention-based copy برای rare word در مقیاس بزرگ قابل اعتماد نبودند)،
- همچنین گاهی ترجمه‌نکردن کامل همه‌ی بخش‌های جمله‌ی ورودی.

(۳) مکانیزم توجه را توضیح دهید.

در GNMT مدل برای هر گام decoding به همه‌ی موقعیت‌های جمله‌ی ورودی نگاه می‌کند، برای هر کدام یک امتیاز می‌سازد، آن امتیازها را به وزن‌های احتمالی تبدیل می‌کند و سپس یک context vector به‌صورت میانگین وزن‌دار از بردارهای encoder

می‌سازد. این کار کمک می‌کند decoder روی بخش‌های مهم ورودی در هر لحظه تمرکز کند.

۴) Robustness در مقاله یعنی چه؟

اینجا robustness یعنی اینکه مدل در برابر ورودی‌های متنوع و به‌خصوص کلمات نادر/ساختارهای دشوار، پایدارتر و قابل‌اعتمادتر عمل کند. مقاله صراحتاً ضعف NMT‌های قدیمی را در robustness، به‌ویژه برای rare words، مطرح می‌کند و GNMT را پاسخی به همین مشکل معرفی می‌کند.

۵) روش معرفی‌شده و دلیل اجزای آن را شرح دهید.

روش مقاله یک encoder-decoder عمیق LSTM با ۸ لایه encoder و ۸ لایه decoder است که با attention و residual connections ساخته شده و برای واژگان باز از wordpiece استفاده می‌کند. در decoding هم از length normalization و coverage penalty در beam search بهره می‌گیرد. همچنین از parallelism و quantized inference استفاده می‌کند. دلیل این اجزا این است که:

- LSTM عمیق برای یادگیری الگوهای پیچیده‌ی ترجمه لازم است.
- Attention کمک می‌کند مدل روی بخش‌های مهم جمله‌ی ورودی تمرکز کند.
- Residual connections کمک می‌کنند گرادیان بهتر عبور کند و شبکه‌های عمیق قابل‌آموزش و پایدارتر شوند.

- **Wordpiece** ها مشکل واژگان باز و کلمات نادر را بهتر مدیریت می کنند و بین دقت سطح کلمه و انعطاف پذیری سطح کاراکتر تعادل ایجاد می کنند.
- **Length normalization** و **coverage penalty** هم در decoding و beam search برای بهتر شدن کیفیت خروجی، باعث می شوند به ترتیب ترجمه های خیلی کوتاه یا ناقص کمتر ترجیح داده شوند و همه ی اجزای جمله ی ورودی را پوشش دهد.

۶) چرا از residual connections استفاده شده است؟

چون در شبکه های عمیق LSTM بدون residual، آموزش سخت می شود و گرادین می تواند ضعیف یا ناپایدار شود. پس این اتصالات جریان گرادین را بهتر می کنند و آموزش لایه های زیاد را ممکن می سازند. همچنین مقاله توضیح می دهد که بدون residual connection، شبکه های عمیق تر از حدود ۶ تا ۸ لایه سخت تر آموزش می بینند، اما residual ها این مشکل را تا حد زیادی کاهش می دهند.

۷) نتیجه مقاله چیست؟

مقاله نشان می دهد GNMT روی بنچمارک های WMT و نیز داده های تولیدی گوگل به نتایج بسیار رقابتی یا برتر می رسد و در ارزیابی انسانی، نسبت به سیستم phrase-based قبلی، خطاها را به طور متوسط حدود ۶۰٪ کم می کند. همچنین wordpiece modeling، quantization و beam search اصلاح شده از اجزای کلیدی موفقیت اند.

٤- مراجع

- [1] <https://arxiv.org/pdf/1609.08144>
- [2] <https://github.com/>